

HTML, CSS и DOM

Как устроена веб-страница

Frontend-разработчик

Одна страница, несколько задач

- Разметить список задач и форму
- Оформить страницу и расположить элементы
- Понять дерево документа в памяти браузера
- Увидеть, как действие пользователя меняет страницу

Сначала устройство страницы, затем подробное изучение JavaScript

Что за что отвечает

Технология	Назначение
HTML	Содержание, структура и смысл элементов
CSS	Оформление и расположение
DOM	Дерево объектов документа и API для работы с ним
JavaScript	Программная логика и обработка событий

DOM создаёт браузер при разборе документа

Первый HTML-документ

```
<!doctype html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Мои задачи</title>
</head>
<body>
  <h1>Мои задачи</h1>
  <p>Сегодня изучаем веб.</p>
</body>
</html>
```

Сохраняем в index.html.

head содержит сведения о документе.

body содержит содержимое страницы.

title задаёт название вкладки, h1 задаёт заголовок в документе

Элемент, тег и атрибут

```
<a href="/courses">Все курсы</a>

<p class="note">Встречаемся в 10:00</p>

<img src="cover.jpg" alt="Обложка книги">
```

a, p, img называют элементы.

href, class, src, alt задают атрибуты.

У img нет закрывающего тега.

Элемент включает разметку, атрибуты и, когда допустимо, содержимое

Вложенность элементов

```
<main>
  <h1>Мои задачи</h1>
  <ul>
    <li>Прочитать материал</li>
    <li>Сделать практику</li>
  </ul>
</main>
```

main содержит h1 и ul.

ul содержит два li.

Отступы помогают видеть вложенность.

Вложенность задаёт отношения родителя и потомков

Семантика HTML

Элемент	Что обозначает
h1 … h6, p	Заголовки разных уровней и абзацы
ul / ol, li	Неупорядоченный / упорядоченный список
main, nav, article	Основное содержимое, навигация, самостоятельный материал
div, span	Универсальные контейнеры без специального смысла

Размер заголовка задаётся CSS, уровень отражает структуру

Ссылка и кнопка

```
<a href="/tasks">Открыть задачи</a>

<button type="button">Удалить задачу</button>

<button type="submit">Отправить форму</button>
```

Ссылка ведёт к ресурсу или месту в документе.

Кнопка запускает действие.

В форме важно явно выбрать её type.

Нативные элементы уже поддерживают привычное управление с клавиатуры

Форма и подпись поля

```
<form action="/search" method="get">
  <label for="query">Поиск задач</label>
  <input id="query" name="q" required>
  <button type="submit">Найти</button>
</form>
```

for связывает label с id поля.

name задаёт имя при отправке.

required требует заполнения.

HTML-форма может отправить данные без JavaScript

Доступность начинается с разметки

- Кнопки и ссылки доступны с клавиатуры
- Подпись label объясняет назначение поля
- alt передаёт смысл содержательного изображения
- Видимый фокус помогает понять, где находится пользователь

Tab перемещает фокус между доступными элементами управления

Подключение CSS

```
<!-- Внутри head -->
<link rel="stylesheet" href="./styles.css">

/* Файл styles.css */
h1 {
  color: #c2410c;
  font-size: 2rem;
}
```

h1 здесь селектор.

color и font-size называют свойства.

После двоеточия находится значение.

Одно правило может оформить много элементов

Основные селекторы

Селектор	Что выбирает
p	Все элементы p
.task	Элементы с class="task"
#tasks	Элемент с id="tasks"
.task button	Кнопки внутри .task на любой глубине
button:hover	Кнопку под указателем

class можно повторять, id должен быть уникален в документе

Какой цвет получит текст?

```
<!-- HTML -->
<p class="note">Сдать практику</p>

/* CSS: один файл, обычные правила */
.note { color: orange; }
p     { color: blue; }
.note { color: green; }
```

Три правила подходят одному элементу.

Последняя строка всегда побеждает?

Сначала предположите результат.

Условия примера: одинаковый источник и слой, без !important

Каскад и специфичность

```
.note { color: orange; }
p     { color: blue; }
.note { color: green; }

/* Результат: green */
```

Класс специфичнее селектора элемента.

У двух .note специфичность равна.

Тогда побеждает более позднее правило.

Порядок записи решает конфликт только после предыдущих критериев

Наследование свойств

```
body {
  color: #111827;
  font-family: sans-serif;
}

.note {
  color: #c2410c;
}
```

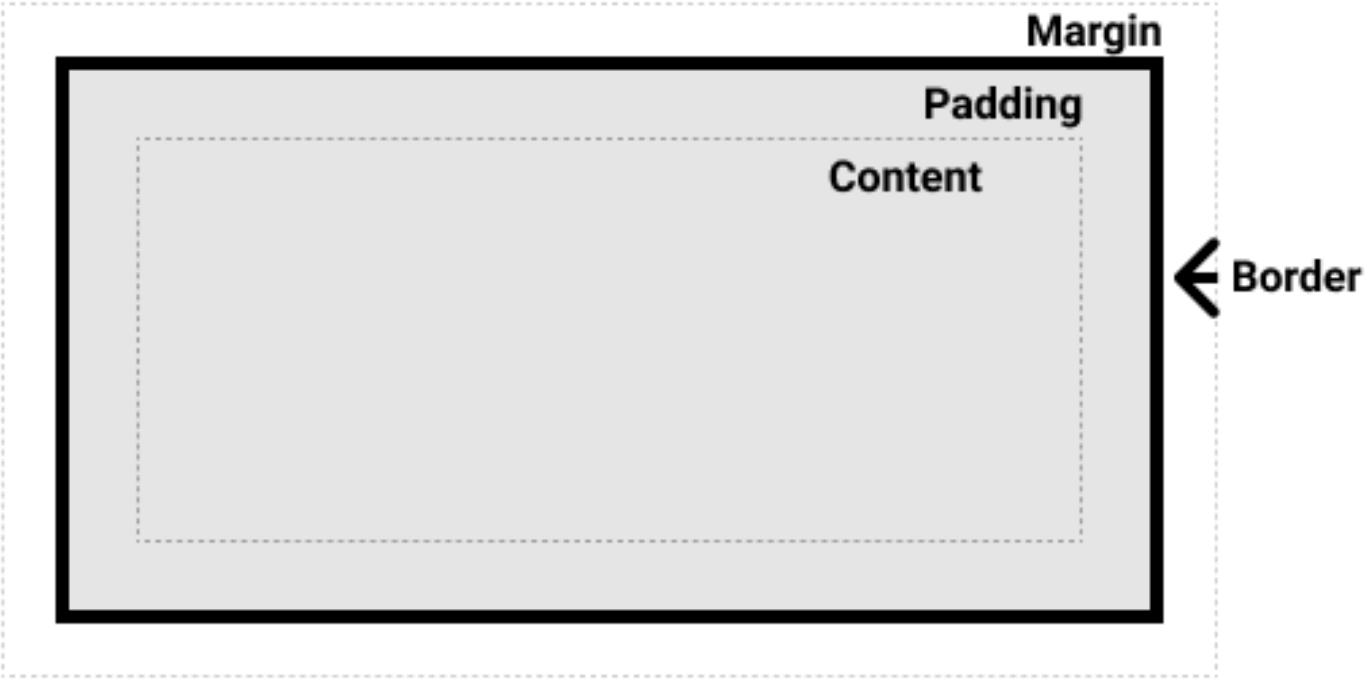
Текст обычно наследует color и font-family.

Собственное правило может изменить значение.

margin и border по умолчанию не наследуются.

Наследование зависит от конкретного свойства

Блочная модель: из чего состоит размер



content: содержимое

padding: внутренний отступ

border: граница

margin: внешний отступ

У размера содержимого и внешнего размера блока разные значения

Вопрос: сколько места занимает блок?

```
.card {
  box-sizing: content-box;
  width: 200px;
  padding: 20px;
  border: 2px solid;
  margin: 10px;
}
```

Какова ширина до внешних краёв рамки?

Что изменится при border-box?

Считаем горизонтальные размеры; отступы заданы со всех сторон

content-box и border-box

При width: 200px	content-box	border-box
Содержимое	200 px	156 px
Padding слева и справа	40 px	40 px
Рамка слева и справа	4 px	4 px
Ширина с рамкой	244 px	200 px

margin остаётся снаружи в обоих случаях

Предсказуемые размеры

```
*, *::before, *::after {
  box-sizing: border-box;
}

main {
  max-width: 48rem;
  margin-inline: auto;
  padding: 1rem;
}
```

border-box включает padding и border в width.

max-width ограничивает ширину.

auto центрирует более узкий блок по горизонтали.

rem зависит от размера шрифта корневого элемента

Обычный поток документа

```
<h1>Задачи</h1>
<p>Сегодня нужно <strong>две</strong> вещи.</p>
<ul>
  <li>Прочитать</li>
  <li>Попробовать</li>
</ul>
```

Блочные элементы обычно идут друг под другом.

Строчные располагаются в строках текста.

CSS display может изменить поведение.

Расположение начинается с обычного потока, без координат вручную

Flexbox: элементы в строке

```
.toolbar {
  display: flex;
  gap: 12px;
  align-items: center;
  justify-content: space-between;
  flex-wrap: wrap;
}
```

Прямые дочерние элементы становятся flex-элементами.

gap задаёт промежуток.

wrap разрешает перенос.

Для row главная ось идёт вдоль строки, поперечная поперёк

Grid: строки и столбцы

```
.tasks {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 16px;
}
```

Два столбца делят доступное пространство.

Следующие элементы переходят на новую строку.

Подходит для сетки карточек.

Flexbox удобен для ряда, Grid задаёт сетку по двум осям

Адаптивность

```
<!-- Внутри head -->
<meta name="viewport"
      content="width=device-width, initial-scale=1">

/* CSS */
.tasks { display: grid; gap: 16px; }

@media (min-width: 48rem) {
  .tasks { grid-template-columns: 1fr 1fr; }
}
```

По умолчанию один столбец.

На достаточно широком экране два.

Порог выбираем по содержимому.

Проверяем узкий экран и увеличение масштаба

Состояния элемента в CSS

```
button:hover {
  background: #9a3412;
}
button:focus-visible {
  outline: 3px solid #0369a1;
  outline-offset: 3px;
}
button:disabled {
  opacity: 0.5;
}
```

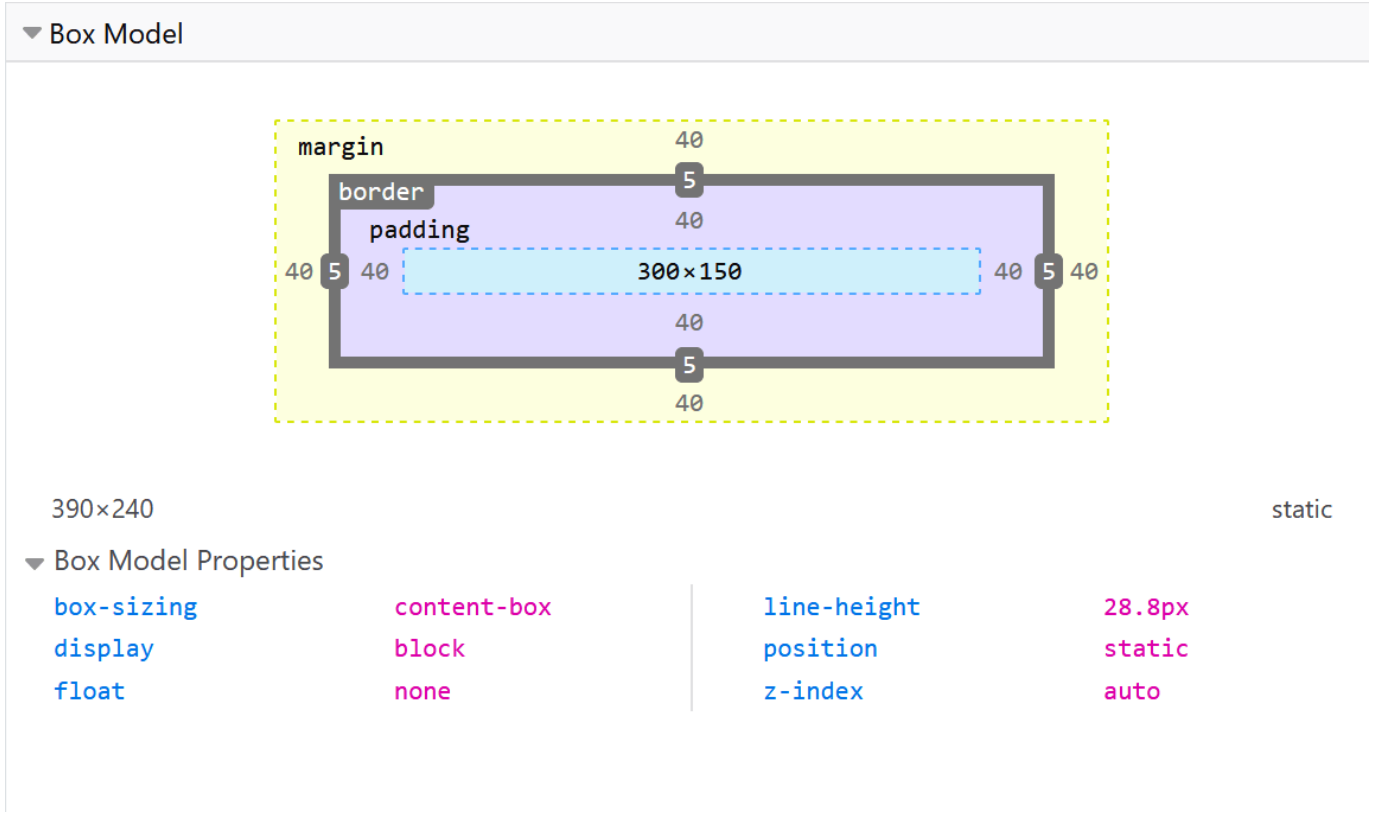
hover реагирует на указатель.

focus-visible помогает видеть клавиатурный фокус.

disabled отражает отключённое состояние.

Интерфейс должен оставаться понятным без мыши

DevTools: почему стиль не применяется



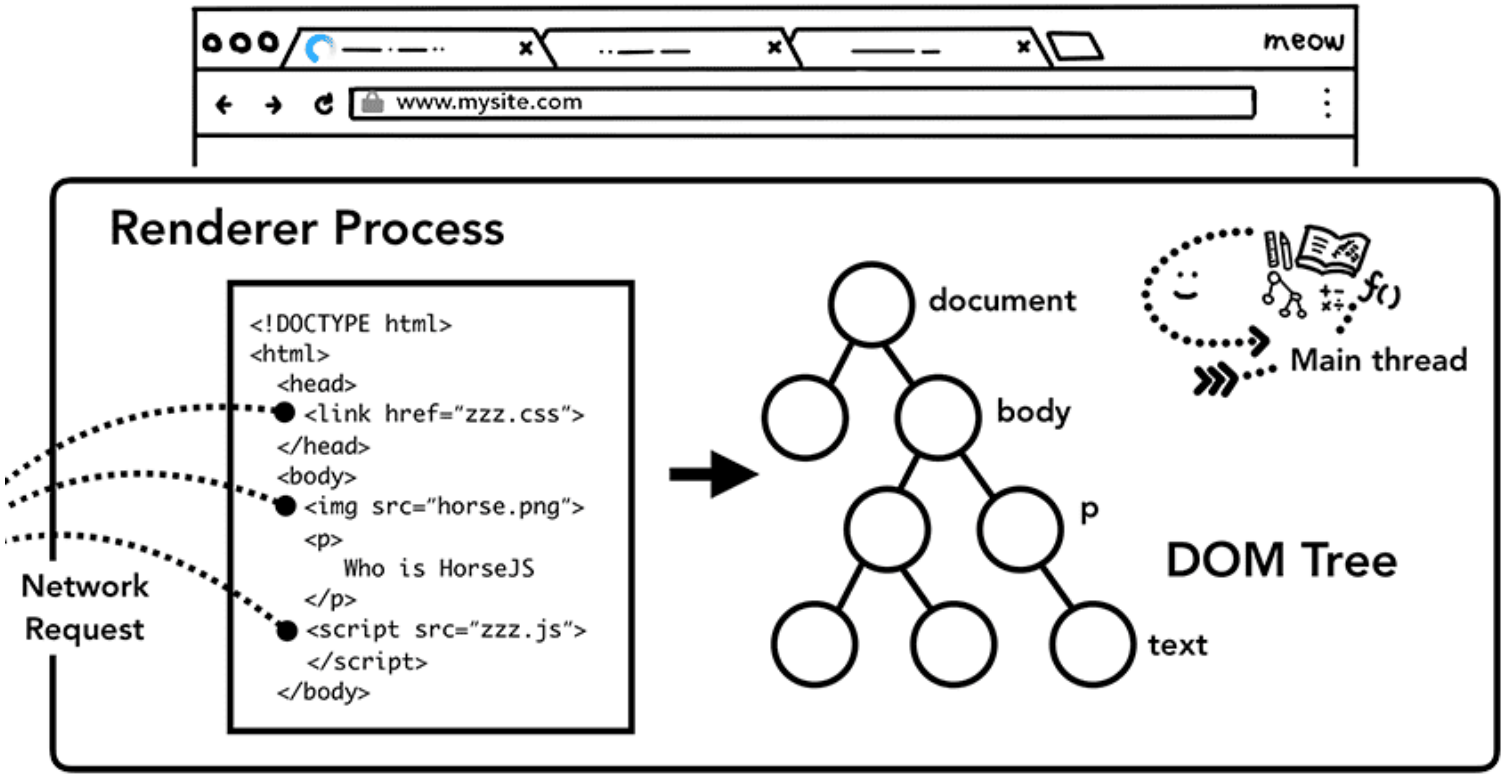
Выбираем элемент.

Смотрим подходящие правила.

Проверяем итоговые значения и размеры.

Изменения в инспекторе обычно исчезают после перезагрузки

DOM: документ в памяти браузера



DOM: Document Object Model.

Узлы образуют дерево.

JavaScript обращается к узлам через API.

Исходный HTML и текущее дерево DOM могут различаться

HTML-файл и живой документ

- Браузер разбирает разметку и создаёт узлы
- Парсер может исправлять ошибочную структуру
- Скрипт может добавлять, удалять и менять узлы
- Elements / Inspector показывает текущее дерево

Изменение DOM не переписывает исходный файл на сервере

Первое изменение через DOM API

```
<!-- HTML -->
<h1 id="title">Мои задачи</h1>

// JavaScript в Console на этой странице
 document.querySelector("#title")
   .textContent = "План на сегодня";
```

document представляет документ.

querySelector ищет первый подходящий элемент.

textContent меняет текст.

Это знакомство с API; синтаксис JavaScript изучим отдельно

CSS-класс как состояние

```
<!-- HTML -->
<li id="task" class="task">Прочитать</li>

/* CSS */
.done { text-decoration: line-through; }

// JavaScript
 document.querySelector("#task")
   .classList.toggle("done");
```

CSS описывает вид выполненной задачи.

JavaScript добавляет или убирает класс.

toggle переключает его наличие.

Поведение меняет состояние элемента, CSS задаёт оформление

Событие и обработчик

```
const button = document.querySelector("#mark");
const task = document.querySelector("#task");

button.addEventListener("click", function () {
  task.classList.toggle("done");
});
```

На странице есть кнопка с id="mark" и задача с id="task".

Браузер вызывает функцию при click.

addEventListener регистрирует действие, которое выполнится позже

Всплытие событий

```
<ul id="tasks">
  <li>
    <button type="button">Удалить</button>
  </li>
</ul>
```

click сначала достигает кнопки.

На фазе всплытия обработчики могут сработать у li и ul, затем выше.

Один обработчик списка может обслужить его кнопки.

Делегирование использует всплытие и определение источника события

Отправка формы и действие по умолчанию

```
const form = document.querySelector("#add-task");

form.addEventListener("submit", function (event) {
  event.preventDefault();
  // Здесь позже добавим задачу в список
});
```

Обычно форма запускает отправку данных.

preventDefault отменяет действие по умолчанию.

submit обрабатывает и отправку клавишей Enter.

preventDefault не останавливает всплытие события

Подключение скрипта и готовность DOM

- Браузер строит дерево по мере чтения HTML
- Скрипт не найдёт элемент, который ещё не появился
- Внешний классический script с defer выполнится после разбора HTML
- Скрипты type="module" по умолчанию также откладывают выполнение

Пример: <script src="./main.js" defer></script>

Текст пользователя на странице

- textContent вставляет значение как текст
- innerHTML разбирает строку как HTML-разметку
- Непроверенная разметка может создать опасные элементы
- Для названия задачи достаточно textContent

Пользовательский ввод по умолчанию выводим как текст

Мини-задача: что изменится после обновления?

- В Elements изменили заголовок
- В Styles поменяли цвет кнопки
- В Console добавили класс done
- Затем перезагрузили страницу

Какие изменения сохранятся и где их нужно записать?

Демонстрация после лекции

- Открыть index.html и изучить структуру
- Подключить styles.css и оформить список
- Сравнить страницу на широком и узком экране
- Изменить элемент в DevTools и перезагрузить
- Показать переключение класса готовым обработчиком

Один проект: список задач с формой добавления

Практика: страница «Мои задачи»

- Заголовок, список из трёх задач и форма с label
- CSS в отдельном файле, понятные классы
- Box-sizing: border-box, отступы и flex-панель
- На узком экране нет горизонтальной прокрутки
- Кнопки доступны с клавиатуры, фокус виден

Дополнительно: класс done меняет вид выполненной задачи

Что важно унести с собой

- HTML задаёт смысл и структуру
- CSS рассчитывает оформление по правилам каскада
- Размер блока включает несколько областей
- DOM представляет документ объектами в памяти
- JavaScript сможет менять эти объекты и реагировать на события

Следующая тема: сам язык JavaScript

Источники и дальнейшее чтение

- MDN: структура HTML и формы
- MDN: каскад, блочная модель и компоновка
- MDN: DOM API и обработка событий
- Chrome for Developers: устройство браузера

Ссылки на конкретные материалы находятся в заметках слайдов